

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
«Работа с БД в СУБД MongoDB»**

Обучающийся Заботкина Марина Александровна
Факультет прикладной информатики
Группа К3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Цель работы

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Практическое задание

1. Выполнить CRUD-операции в СУБД MongoDB, вставку данных и выборку данных.
2. Запросы к БД MongoDB. Выборка данных. Вложенные объекты. Использование курсоров. Агрегированные запросы. Изменение данных.
3. Ссылки и работа с индексами в базе данных MongoDB

Выполнение

Задание 2.1.1

1. *Создайте базу данных learn.*
2. *Заполните коллекцию единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm',  
vampires: 63});  
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f',  
vampires: 43});  
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender:  
'm', vampires: 182});  
db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm',  
vampires: 99});  
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550,  
gender: 'f', vampires: 80});  
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender:  
'f', vampires: 40});  
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm',  
vampires: 39});  
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm',  
vampires: 2});  
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender:  
'f', vampires: 33});  
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender:  
'm', vampires: 54});  
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender:  
'f'});
```

3. *Используя второй способ, вставьте в коллекцию единорогов документ:*

```
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```

4. *Проверьте содержимое коллекции с помощью метода find.*

Создание базы данных и заполнение коллекции:

use learn

```
db.unicorns.insertMany([ {name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63}, {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43}, {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182}, {name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99}, {name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80}, {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40}, {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39}, {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2}, {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33}, {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54}, {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'} ])
```

```
learn> use learn
already on db learn
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
| db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
| db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
| db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
| db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
| db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
| db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
| db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
| db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
| db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
| db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0a5662dd8c662d63abc11f') }
}
```

Создание документа и вставка в бд:

```
document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```

```
db.unicorns.insertOne(document)
```

```
learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a0a57f4dd8c662d63abc120')
}
```

Содержание коллекции:

```
db.unicorns.find()
```

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0a5662dd8c662d63abc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc117'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc118'),
    name: 'Rooooooodles',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0a57f4dd8c662d63abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
```

Задание 2.2.1

1. Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.
2. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

Список самцов с сортировкой по имени:

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a0a57f4dd8c662d63abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11b'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc118'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc117'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
```

Список первых трёх самок с сортировкой по имени:

```
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
```

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a0a5662dd8c662d63abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11d'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

Первая самка, которая любит carrot с помощью findOne:

```
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
```

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a0a5662dd8c662d63abc116'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

Первая самка, которая любит carrot с помощью find и limit:

```
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
```

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a0a5662dd8c662d63abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
[
  {
    _id: ObjectId('6a0a57f4dd8c662d63abc120'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc115'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11b'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11e'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },

```

```

  {
    _id: ObjectId('6a0a5662dd8c662d63abc11c'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc118'),
    name: 'Roooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc117'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  }
]
```

Задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
db.unicorns.find().sort({$natural:-1})
```

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a0a57f4dd8c662d63abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },

```

```

  {
    _id: ObjectId('6a0a5662dd8c662d63abc11d'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11c'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11b'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },

```

```

{
  _id: ObjectId('6a0a5662dd8c662d63abc11a'),
  name: 'Ayna',
  loves: [ 'strawberry', 'lemon' ],
  weight: 733,
  gender: 'f',
  vampires: 40
},
{
  _id: ObjectId('6a0a5662dd8c662d63abc119'),
  name: 'Solnara',
  loves: [ 'apple', 'carrot', 'chocolate' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  _id: ObjectId('6a0a5662dd8c662d63abc118'),
  name: 'Rooodles',
  loves: [ 'apple' ],
  weight: 575,
  gender: 'm',
  vampires: 99
},

```

```

{
  _id: ObjectId('6a0a5662dd8c662d63abc117'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182
},
{
  _id: ObjectId('6a0a5662dd8c662d63abc116'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a0a5662dd8c662d63abc115'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
]

```

Задание 2.2.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
db.unicorns.find({}, {loves: {$slice: 1}, _id: 0})
```



```
learn> db.unicorns.find({}, {loves: {$slice: 1}, _id: 0})
[
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: [ 'energon' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
]
```

```
{
  name: 'Solnara',
  loves: [ 'apple' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  name: 'Ayna',
  loves: [ 'strawberry' ],
  weight: 733,
  gender: 'f',
  vampires: 40
},
{
  name: 'Kenny',
  loves: [ 'grape' ],
  weight: 690,
  gender: 'm',
  vampires: 39
},
{
  name: 'Raleigh',
  loves: [ 'apple' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
]
```

```
{
  name: 'Leia',
  loves: [ 'apple' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  name: 'Pilot',
  loves: [ 'apple' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  name: 'Nimue', loves: [ 'grape' ], weight: 540, gender: 'f' },
{
  name: 'Dunx',
  loves: [ 'grape' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
]
```

Задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
```

```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
db.unicorns.find({vampires: {$exists: false}})
```

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
```

```
learn> db.unicorns.find({ gender: 'm' }, { name: 1, loves: { $slice: 1 }, _id: 0 }).sort({ name: 1 })
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Задание 3.1.1:

1. *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",
populatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: [""],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
populatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
party: "I"}}

{name: "Portland",
populatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
party: "D"}}
```

2. *Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I") . Вывести только название города и информацию о мэре.*

3. *Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует) . Вывести только название города и информацию о мэре.*

Создание коллекции towns:

```
db.towns.insert ([ {name: "Punxsutawney", population: 6200, last_census: ISODate("2008-01-31"), famous_for: ["phil the groundhog"], mayor: {name: "Jim Wehrle"}}, {name: "New York", population: 22200000, last_census: ISODate("2009-07-31"), famous_for: ["status of liberty", "food"], mayor: {name: "Michael Bloomberg", party: "I"}}, {name: "Portland", population: 528000, last_census: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {name: "Sam Adams", party: "D"}} ] )
```

```
learn> db.towns.insert([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: [""],
    mayor: { name: "Jim Wehrle" }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: { name: "Sam Adams", party: "D" }
  }
])
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0b70cbe88d92be43abc114'),
    '1': ObjectId('6a0b70cbe88d92be43abc115'),
    '2': ObjectId('6a0b70cbe88d92be43abc116')
  }
}
```

Список городов с независимыми мэрами:

```
db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
```

```
learn> db.towns.find({ "mayor.party": "I" }, { name: 1, mayor: 1, _id: 0 })
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
```

Список беспартийных мэров:

```
db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
```

```
learn> db.towns.find({ "mayor.party": { $exists: false } }, { name: 1, mayor: 1, _id: 0 })
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
```

Задание 3.1.2:

1. Сформировать функцию для вывода списка самцов единорогов.
2. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
3. Вывести результат, используя *forEach*.

```
var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2);
```

```
cursor.forEach(function(unicorn) { print(unicorn.name); })
```

```
learn> var cursor = db.unicorns.find({ gender: 'm' }).sort({ name: 1 }).limit(2);

learn> cursor.forEach(function(unicorn) { print(unicorn.name); });
Dunx
Horny
```

Задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
```

```
learn> db.unicorns.find({ gender: 'f', weight: { $gte: 500, $lte: 600 } }).count()
2
```

Задание 3.2.2:

Вывести список предпочтений.

```
db.unicorns.distinct("loves")
```

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}])
```

```
learn> db.unicorns.aggregate([{$group: { _id: "$gender", count: { $sum: 1 } } }])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
```

Задание 3.3.1:

1. *Выполнить команду:*

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

2. *Проверить содержимое коллекции unicorns.*

```
db.unicorns.insert ({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
```

(в новых версиях save убрали)

```
learn> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
TypeError: db.unicorns.save is not a function
learn> db.unicorns.insert({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a0b82d3e88d92be43abc117') }
}
```

Содержимое коллекции unicorns:

```
db.unicorns.find({name: 'Barney'})
```

```
learn> db.unicorns.find({name: 'Barney'})
[
  {
    _id: ObjectId('6a0b82d3e88d92be43abc117'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

Задание 3.3.2:

1. Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции unicorns.

```
db.unicorns.updateOne({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
```

```
db.unicorns.find({name: 'Ayna'})
```

```
learn> db.unicorns.updateOne({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: 'Ayna' })
[
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
```

Задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции unicorns.

```
db.unicorns.updateOne({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
```

```
db.unicorns.find({name: 'Raleigh'})
```

```
learn> db.unicorns.updateOne({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11c'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Задание 3.3.4:

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.
2. Проверить содержимое коллекции unicorns.

```
db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
```

```
db.unicorns.find({gender: 'm'})
```

```
learn> db.unicorns.updateMany({ gender: 'm' }, { $inc: { vampires: 5 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find({ gender: 'm' })
[
  {
    _id: ObjectId('6a0a5662dd8c662d63abc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc117'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a0a5662dd8c662d63abc118'),
```

Задание 3.3.5:

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции `towns`.

```
db.towns.updateOne({name: "Portland"}, {$unset: {"mayor.party": 1}})
```

```
db.towns.find({name: "Portland"})
```

```
learn> db.towns.updateOne({ name: "Portland" }, { $unset: { "mayor.party": 1 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({ name: "Portland" })
[
  {
    _id: ObjectId('6a0b70cbe88d92be43abc116'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Задание 3.3.6:

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.
2. Проверить содержимое коллекции `unicorns`.

```
db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
```

```
db.unicorns.find({name: 'Pilot'})
```

```
learn> db.unicorns.updateOne({ name: 'Pilot' }, { $push: { loves: 'chocolate' } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: 'Pilot' })
[
  {
    _id: ObjectId('6a0a5662dd8c662d63abc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```


Задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

```
db.unicorns.updateOne({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})  
db.unicorns.find({name: 'Aurora'})
```

```
learn> db.unicorns.updateOne({ name: 'Aurora' }, { $addToSet: { loves: { $each: ['sugar', 'lemon'] } } })  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
learn> db.unicorns.find({ name: 'Aurora' })  
[  
  {  
    _id: ObjectId('6a0a5662dd8c662d63abc116'),  
    name: 'Aurora',  
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],  
    weight: 450,  
    gender: 'f',  
    vampires: 43  
  }  
]
```

Задание 3.4.1:

1. Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",  
popujatiuon: 6200,  
last_sensus: ISODate("2008-01-31"),  
famous_for: ["phil the groundhog"],  
mayor: {  
  name: "Jim Wehrle"  
}}
```

```
{name: "New York",  
popujatiuon: 22200000,  
last_sensus: ISODate("2009-07-31"),  
famous_for: ["status of liberty", "food"],  
mayor: {  
  name: "Michael Bloomberg",  
  party: "I"}}}
```

```
{name: "Portland",  
popujatiuon: 528000,  
last_sensus: ISODate("2009-07-20"),  
famous_for: ["beer", "food"],  
mayor: {  
  name: "Sam Adams",  
  party: "D"}}}
```

2. Удалите документы с беспартийными мэрами.
3. Проверьте содержание коллекции.
4. Очистите коллекцию.
5. Просмотрите список доступных коллекций.

Создание коллекции towns:

```
learn> db.towns.insert([
  {
    name: "Punxsutawney ",
    popujatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    popujatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    popujatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
```

Удаление беспартийных и проверка содержания коллекции:

```
db.towns.remove({"mayor.party": {$exists: false}})
```

```
db.towns.find()
```

```
learn> db.towns.remove({ "mayor.party": { $exists: false } })
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0b8fea3894cac9d1abc115'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0b8fea3894cac9d1abc116'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

Очистка коллекции и просмотр списка коллекций:

```
db.towns.remove({})
```

```
show collections
```

```
learn> db.towns.remove({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
```

При выполнении команды `remove` коллекция очищается от документов, но остаётся существовать в базе данных.

Задание 4.1.1:

1. *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*
2. *Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.*
3. *Проверьте содержание коллекции единорогов.*
4. *Содержание коллекции единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
```

```
db.unicorns.insert({name: 'Roooooodles', 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
```

```
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
```

```
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
```

```
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
```

```
db.unicorns.insert ({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

```
db.unicorns.insert {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```

Создание коллекции зон обитания:

```
db.zones.insertMany([ { _id: "forest", name: "Enchanted Forest", description: "A dense forest with eternal twilight"}, { _id: "mountains", name: "High Mountains", description: "Snow-capped peaks where rare species live" } ])
```

```
learn> db.zones.insertMany([
|   { _id: "forest", name: "Enchanted Forest", description: "A dense forest with eternal twilight" },
|   { _id: "mountains", name: "High Mountains", description: "Snow-capped peaks where rare species live" }
| ])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountains' }
}
```

Установка ссылок:

```
db.unicorns.updateOne({name: 'Horny'}, {$set: {zone: {$ref: "zones", $id: "forest"}}})
```

```
db.unicorns.updateOne({name: 'Aurora'}, {$set: {zone: {$ref: "zones", $id: "mountains"}}})
```

```
learn> db.unicorns.updateOne({name: 'Horny'}, {$set: {zone: {$ref: "zones", $id: "forest"}}})
| db.unicorns.updateOne({name: 'Aurora'}, {$set: {zone: {$ref: "zones", $id: "mountains"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Проверка содержание коллекции:

```
db.unicorns.find({zone: {$exists: true}})
```

```
learn> db.unicorns.find({ zone: { $exists: true } })
[
  {
    _id: ObjectId('6a0b93173894cac9d1abc117'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63,
    zone: DBRef('zones', 'forest')
  },
  {
    _id: ObjectId('6a0b93173894cac9d1abc118'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    zone: DBRef('zones', 'mountains')
  }
]
```

Задание 4.2.1:

1. Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

2. Содержание коллекции единорогов `unicorns`:

```
db.unicorns.insert({name: 'Horny', dob: new Date(1992,2,13,7,47), loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', dob: new Date(1991, 0, 24, 13, 0), loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', dob: new Date(1973, 1, 9, 22, 10), loves: ['energion', 'redbull'], weight: 984, gender: 'm', vampires: 182});
```

```
db.unicorns.insert({name: 'Roooooodles', dob: new Date(1979, 7, 18, 18, 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', dob: new Date(1985, 6, 4, 2, 1), loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
```

```
db.unicorns.insert({name: 'Ayna', dob: new Date(1998, 2, 7, 8, 30), loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
```

```
db.unicorns.insert({name: 'Kenny', dob: new Date(1997, 6, 1, 10, 42), loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', dob: new Date(2005, 4, 3, 0, 57), loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', dob: new Date(2001, 9, 8, 14, 53), loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
```

```
db.unicorns.createIndex({name: 1}, {unique: true})
```

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true })
name_1
```

Создание индекса прошло успешно, что подтверждает уникальность имен единорогов в коллекции.

Задание 4.3.1:

1. Получите информацию о всех индексах коллекции `unicorns`.

2. Удалите все индексы, кроме индекса для идентификатора.

3. Попробуйте удалить индекс для идентификатора.

```
db.unicorns.getIndexes()
```

```
db.unicorns.dropIndex("name_1")
```

```
db.unicorns.dropIndex("_id_")
```

```

learn> db.unicorns.createIndex({ name: 1 }, { unique: true })
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index

```

Попытка удаления индекса `_id_` завершилась неудачно, так как данный индекс является обязательным для работы MongoDB и обеспечивает уникальность идентификаторов документов.

Задание 4.4.1:

1. *Создайте объемную коллекцию `numbers`, задействовав курсор:*

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```
2. *Выберите последних четыре документа.*
3. *Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)*
4. *Создайте индекс для ключа `value`.*
5. *Получите информацию о всех индексах коллекции `numbers`.*
6. *Выполните запрос 2.*
7. *Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?*
8. *Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?*

Создание коллекции `numbers`:

```
for(i = 0; i < 100000; i++){db.numbers.insertOne({value: i})}
```

```
learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a0ba1645a05a29d62ad47b3') }
}
learn> db.numbers.find()
[
  { _id: ObjectId('6a0ba0ea5a05a29d62abc114'), value: 0 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc115'), value: 1 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc116'), value: 2 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc117'), value: 3 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc118'), value: 4 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc119'), value: 5 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc11a'), value: 6 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc11b'), value: 7 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc11c'), value: 8 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc11d'), value: 9 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc11e'), value: 10 },
  { _id: ObjectId('6a0ba0ea5a05a29d62abc11f'), value: 11 }
]
```

Выбор последних четырёх документов без индекса:

```
db.numbers.explain("executionStats").find().sort({$natural:-1}).limit(4)
```

```
learn> db.numbers.explain("executionStats").find().sort({$natural: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: '4E45B701',
    planCacheShapeHash: '4E45B701',
    planCacheKey: '36FF8BFB',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'COLLSCAN',
        nss: 'learn.numbers',
        direction: 'backward'
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 5,
    totalKeysExamined: 0,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,

```

Выборка последних четырёх документов заняла 5 мс.

Создание индекса для ключа value и получение списка всех индексов коллекции:

```
db.numbers.createIndex({value: 1})
```

```
db.numbers.getIndex()
```

```
learn> db.numbers.createIndex({ value: 1 })
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

Выбор последних четырёх документов с индексом:

```
db.numbers.explain("executionStats").find().sort({value:-1}).limit(4)
```

```
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 4,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nss: 'learn.numbers',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'learn.numbers',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 8,
  }
}
```

Выборка четырёх документов с индексом заняло 8 мс, но в это время входит поиск документов и оптимизация (4 мс). То есть сам поиск занял всего 4 мс, что чуть быстрее, чем поиск без индекса. Чем больше база данных, тем более быстрым (в сравнении с поиском без индекса) будет поиск с индексом.

Выводы:

В ходе выполнения лабораторной работы были освоены базовые CRUD-операции: получены практические навыки создания, чтения, обновления и удаления документов в среде NoSQL. Была отработаны приемы манипуляции массивами (операторы `$push`, `$addToSet`, `$slice`) и вложенными объектами. На практике применены инструменты для вычисления статистических данных (методы `count`, `distinct`, `aggregate`), а также принципы группировки данных по заданным критериям. Также были освоены способы организации логических связей между коллекциями с помощью ссылок.

С помощью метода `explain()` проведен сравнительный анализ производительности запросов. Без индекса программе приходится перебирать все документы по порядку, а с индексом она сразу находит нужные данные. Это делает поиск почти мгновенным, даже если в базе сотни тысяч записей.